

censo-escolar — análise dos microdados do Censo Escolar (INEP)

wgn com apoio do Claude

August 13, 2026

Contents

Projeto Python para análise dos microdados do **Censo Escolar da Educação Básica**, publicados anualmente pelo INEP.

O projeto foi montado em torno de um requisito específico: **a mesma base de código ser editável no org-mode do Emacs e no Jupyter Notebook**, por pessoas diferentes, ao mesmo tempo.

A seção seguinte explica como isso é resolvido — vale ler antes de mexer em qualquer coisa.

A resposta curta: sim, é possível — em duas camadas

Não existe hoje uma ferramenta que sincronize `.org` e `.ipynb` de forma totalmente automática e sem perdas (o `jupytertext`, que faria isso, não suporta `org-mode`; veja Alternativas consideradas). Mas o problema real se resolve melhor quando é dividido em duas camadas:

Camada 1 — o código de verdade mora no pacote Python

Tudo que é lógica de análise fica em `src/censo_escolar/`: leitura dos CSVs, dicionários de códigos, agregações, gráficos. São arquivos `.py` comuns, versionados no git, testados com `pytest`, sem nenhum vínculo com `org` ou com Jupyter.

Esta é a parte que resolve 90% do problema. Um `.py` é literalmente o mesmo arquivo para os dois ambientes — não há o que

sincronizar. Quem usa Emacs edita com `python-mode`; quem usa Jupyter edita no JupyterLab ou no editor que preferir; o git resolve os conflitos como resolveria em qualquer projeto.

Os documentos de análise ficam **magros**: importam do pacote, chamam funções, mostram tabelas e gráficos, e escrevem a narrativa em volta. Se você se pegar escrevendo uma função dentro de um bloco de código, ela pertence a `src/censo_escolar/`.

Camada 2 — os documentos narrativos são convertidos sob demanda

Para a camada fina que sobra (prosa + chamadas + saídas), o projeto traz um conversor bidirecional próprio: `src/censo_escolar/orgnb.py`.

```
notebooks/01-panorama-escolas.org  <-->  notebooks/01-panorama-escolas.ipynb
```

Você edita o formato que preferir e roda `make sync`. A direção é decidida pelo `mtime`: o arquivo mais recente do par manda no outro.

O conversor preserva o que costuma se perder nesse tipo de conversão:

- **Argumentos de cabeçalho do org-babel** (`:session`, `:results`, `:async`, `:var ...`) sobrevivem à ida e à volta, guardados em `metadata.org` da célula. É exatamente o que o `pandoc` descarta.
- **Saídas e `execution_count`** do notebook não são apagados quando você reconverte a partir do `.org`.
- **Ids das células** são estáveis, então editar uma linha gera um diff de uma linha — e não do arquivo inteiro.
- **Formatação org sem equivalente em markdown** (tags, propriedades, palavras-chave) é reemitida tal e qual, desde que ninguém tenha editado aquele texto do lado do Jupyter.

A ida e volta `org -> ipynb -> org` é **byte a byte idêntica**; há testes que garantem isso (`tests/test_orgnb.py`).

O terceiro elemento: o mesmo kernel nos dois lados

Com o pacote `emacs-jupyter`, os blocos `jupyter-python` do org rodam num **kernel Jupyter de verdade** — o mesmo processo, o mesmo protocolo e o mesmo ambiente que o JupyterLab usa. Não é uma reimplementação do Python dentro do Emacs.

Consequência prática: quem está no Emacs e quem está no Jupyter executam o mesmo código, no mesmo kernel **censo-escolar**, e obtêm os mesmos resultados.

Instalação

Requisitos: Python 3.11, uv e, para o lado Emacs, Emacs 28 com suporte a módulos dinâmicos. O `make` é conveniência, não requisito — todo alvo é uma única chamada ao `uv` ou ao comando `censo` (instalado no `.venv` pelo próprio `uv`), e a seção No Windows mostra o equivalente sem `make`.

Instalando o uv

Se o comando `uv` já existe no seu terminal (`uv --version`), pule esta parte.

O `uv` é quem cria e mantém o `.venv` deste projeto e resolve suas dependências a partir do `pyproject.toml` `=> uv.lock` — substitui, num comando só, o que antes seria `python -m venv + pip install`. Ele não é um pacote Python: instala-se uma vez, fora de qualquer `venv`, com o instalador oficial do projeto Astral:

```
# Linux / macOS
```

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Windows (PowerShell)
```

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Alternativas, se preferir um gerenciador de pacotes que já usa: `pipx install uv`, `pip install uv`, `brew install uv` (macOS) ou `winget install astral-sh.uv` (Windows). A lista completa, sempre atualizada, fica em <https://docs.astral.sh/uv/getting-started/installation/>. Depois de instalar, abra um terminal novo e confira com `uv --version`.

Duas ideias cobrem o `uv` inteiro para os fins deste projeto:

- `uv sync` lê o `pyproject.toml` `=> uv.lock` e deixa o `.venv` igual ao que eles descrevem — cria o `venv` se não existir, instala o que falta, remove o que sobra. É o que `make instalar` chama por baixo.

- `uv run <comando>` roda `<comando>` dentro desse `.venv`, sincronizando antes se preciso. Não precisa ativar o `venv` à mão nem lembrar do caminho de `.venv/bin` (ou `.venv\Scripts` no Windows) — é assim que os exemplos `uv run censo ...` abaixo funcionam.

Instalando o projeto

```
make instalar      # uv sync: cria .venv e instala o pacote em modo editável + extras
make kernel        # registra o kernel Jupyter "censo-escolar"
```

Se preferir fazer à mão, ou não tiver `make`:

```
uv sync
uv run censo kernel
```

No Windows

O pacote Python atravessa sem ajuste: `pathlib` em tudo, nenhuma chamada de shell, `encoding` explícito em toda leitura e escrita. O que não atravessa é o `make`, que não vem instalado no Windows e não acompanha o Git for Windows — mas com `uv` instalado, nenhum alvo do Makefile depende de `make` para funcionar: os mesmos comandos, no PowerShell ou no `cmd.exe`:

```
uv sync                # make instalar (venv + editável + extras)
uv run censo kernel    # make kernel

uv run censo obter 2023    # make dados
uv run censo parquet 2023  # make parquet
uv run censo sync notebooks # make sync
uv run censo lab         # make lab
uv run censo test        # make test
uv run censo lint        # make lint
uv run censo limpar      # make limpar
```

`uv run censo <alvo>` é o mesmo comando que o Makefile chama por baixo, então funciona em Linux e macOS também — não é uma lista à parte para o Windows, é o jeito de rodar isto tudo sem `make` em qualquer sistema. Quem usa WSL, MSYS2 ou Git Bash *com* `make` instalado pode continuar com os alvos `make` normalmente: o Makefile descobre sozinho se o `venv` guardou os executáveis em `bin/` ou em `Scripts/`. Ele pergunta ao disco, não ao sistema operacional — é o que faz acertar até o caso torto de um WSL enxergando um `venv` criado pelo Python do Windows, em que a detecção por sistema operacional escolheria `bin/` e falharia.

Duas armadilhas que não dão erro

Finais de linha. O modo texto do Python traduz `\n` em `\r\n` no Windows. Se a sincronização gravasse assim, cada documento mudaria por inteiro só de atravessar um sistema operacional, e todo commit viraria um diff do arquivo todo — sem ninguém conseguir mais enxergar a mudança de verdade. Já está resolvido nas duas pontas: o `orgnb.py` grava sempre com `\n` (veja `_gravar`) e o `.gitattributes` fixa `eol=lf`. Vale saber que a proteção existe antes de alguém "simplificar" uma das duas — em POSIX, remover qualquer uma delas passa despercebido, porque lá a tradução nunca acontece.

Caminhos longos. Durante a extração, os anexos do ZIP do INEP chegam a ~180 caracteres de caminho relativo, e o Windows corta em 260 por padrão. Em `C:\Users\voce\projetos\` sobra folga; dentro de um `C:\Users\voce\OneDrive - Empresa\Documentos\...` o censo obter estoura no meio da extração. Clone perto da raiz do disco, ou habilite o `LongPathsEnabled` no registro.

Obtendo os dados

```
make dados ANO=2023      # baixa o ZIP e extrai em data/interim/
make parquet ANO=2023    # converte o CSV de escolas para Parquet (leituras ~10x mais r
```

Ou, com mais controle:

```
.venv/bin/censo baixar 2023
.venv/bin/censo extrair 2023
.venv/bin/censo parquet 2023
.venv/bin/censo colunas 2023 --filtro MAT    # lista as colunas do ano
.venv/bin/censo caminhos                    # mostra os diretórios resolvidos
```

Se o download falhar

O INEP muda o endereço dos microdados de tempos em tempos, e a URL embutida em `src/censo_escolar/config.py` pode estar desatualizada. Duas saídas:

```
# 1. informar a URL correta na hora
.venv/bin/censo baixar 2023 --url 'https://download.inep.gov.br/.../{ano}.zip'

# 2. ou fixar via variável de ambiente
export CENSO_ESCOLAR_URL='https://download.inep.gov.br/.../{ano}.zip'
```

Baixar o ZIP manualmente do site do INEP e colocá-lo em `data/raw/microdados_censo_escolar_2023` também funciona — o `censo extrair 2023` segue daí.

HTTP 404 num ano que existe

O INEP não é consistente no nome do arquivo. O pacote de 2025 foi publicado como `microdados_censo_escolar_2025.zip` — com um sublinhado sobrando antes da extensão —, enquanto todos os outros anos seguem `..._{ano}.zip`. O resultado é um 404 num ano que a página do INEP lista normalmente, o que engana: parece que o ano não saiu.

Por isso `config.PADROES_URL` é uma **lista** de padrões, tentados em ordem até um responder — mesma ideia do `_PADROES_ESCOLAS` em `loading.py`, que cobre as variantes de nome do CSV. Se aparecer uma variante nova, acrescente-a lá e ela passa a valer para todos os anos.

Antes de concluir que um ano não foi publicado, confira a página de microdados do INEP: testar o padrão de URL que já temos não detecta um arquivo renomeado.

O ano de 2025 mudou mais coisas além do nome do arquivo — inclusive uma que some com colunas sem avisar. Veja `docs/microdados-2025.org`.

ConnectionResetError / Connection aborted

O servidor do INEP derruba a primeira conexão com alguma frequência, com um RST no meio do handshake TLS. Não é erro de configuração sua: a tentativa seguinte quase sempre passa — dá para confirmar com `curl -I` repetido no mesmo endereço.

O `censo baixar` já sobe com retentativa e backoff (5 tentativas, cobrindo erro de conexão, de leitura e 5xx), então esse tropeço é absorvido sozinho. Se ainda assim estourar, é sinal de instabilidade maior do lado do INEP — espere e tente de novo.

unable to get local issuer certificate / SSLError

O `download.inep.gov.br` envia apenas o certificado folha e **omite a CA intermediária** (RNP ICPEdu GR46 OV TLS CA 2025) que o encadeia até a raiz GlobalSign. Como nem o `certifi` nem o armazenamento do sistema trazem essa intermediária, a verificação falha — o mesmo acontece com `curl` e com o navegador em máquinas limpas.

O projeto resolve isso **sem desligar a verificação**: ao ver o erro de SSL, o `censo baixar` baixa a intermediária a partir do campo "CA Issuers" do

próprio certificado do INEP, concatena com o bundle do `certifi` e repete o download. Você verá a linha:

```
Cadeia de certificados incompleta; baixando a CA intermediária...
```

Para montar o bundle antes da hora, ou refazê-lo depois de uma troca de certificado:

```
make certificados      # grava certs/inep-ca.pem
```

Se você já tem um bundle próprio (uma CA corporativa no meio do caminho, por exemplo), aponte para ele e o projeto usa o seu:

```
export CENSO_ESCOLAR_CA_BUNDLE=/caminho/para/meu-bundle.pem
```

Não use `verify=False`. É a "solução" que aparece primeiro em busca, e ela desliga a verificação de todos os certificados para contornar a falta de um só. O bundle acima resolve o problema real mantendo a checagem de pé.

Onde os dados ficam

Os microdados são grandes demais para o git e estão no `.gitignore`. Para guardá-los em outro disco:

```
export CENSO_ESCOLAR_DATA=/mnt/ext4/dados/censo-escolar
```

Usando no Emacs (org-babel)

1. Carregue a configuração de exemplo (ou copie os trechos para o seu `init.el`):

```
(load-file "emacs/censo-init.el")
```

2. Abra `notebooks/01-panorama-escolas.org`.
3. `C-c C-c` em um bloco executa; `C-c C-v C-b` executa o buffer inteiro.

O `#+property:` no topo do arquivo já define `:session censo :async yes`, então o estado persiste entre blocos, como num notebook, e o Emacs não trava enquanto o pandas trabalha.

Gráficos

Este é o único ponto em que os dois ambientes divergem de fato, e o projeto resolve com uma convenção: **toda função de plot aceita arquivo= e devolve o caminho gravado**. No Jupyter, a figura aparece sozinha; no org, o bloco usa `:results file` e o Emacs insere o link da imagem.

```
#+begin_src jupyter-python :results file
plots.barras(uf, x="SG_UF", y="escolas", arquivo="escolas_por_uf.png")
#+end_src
```

Caminhos relativos vão para `reports/figures/`.

Atalho de sincronização

`C-c n s` no org-mode roda `make sync` na raiz do projeto.

Usando no Jupyter — tutorial

Este é o caminho completo para quem vai **desenvolver e rodar as análises pelo Jupyter**, sem tocar no Emacs. Do zero até o commit.

Passo 1 — ambiente e kernel

```
make instalar      # cria .venv e instala o pacote (editável) + jupyterlab + ipykernel
make kernel        # registra o kernel "censo-escolar" no seu perfil de usuário
```

O `make kernel` grava um kernelspec em `~/.local/share/jupyter/kernels/censo-escolar/` apontando para o Python do `.venv` do projeto. Confira:

```
.venv/bin/jupyter kernelspec list
```

Você deve ver `censo-escolar` na lista. Esse passo é o que faz o notebook enxergar o pacote `censo_escolar` — sem ele, o JupyterLab abre com um Python qualquer e o primeiro `import` falha.

Por que um kernel próprio, e não o python3 padrão?

Porque o pacote está instalado em modo editável dentro do `.venv`. Um kernel de fora do `venv` não vê `censo_escolar`, nem o `pandas/pyarrow` nas versões que os testes usam.

Passo 2 — dados

O notebook de exemplo lê o ano de 2023. Baixe uma vez:

```
make dados ANO=2023      # baixa o ZIP do INEP e extrai em data/interim/
make parquet ANO=2023    # opcional, mas recomendado: leituras ~10x mais rápidas
```

Sem isso, a primeira chamada a `carregar_escolas(2023)` levanta `FileNotFoundError`.
Veja Se o download falhar caso a URL do INEP tenha mudado.

Passo 3 — abrir o JupyterLab

```
make lab
```

Equivale a `.venv/bin/jupyter lab notebooks`. Duas variações úteis:

```
.venv/bin/jupyter lab notebooks --no-browser --port 8899  # em servidor/SSH
.venv/bin/jupyter lab                                     # abrindo na raiz do projeto
```

Se os `.ipynb` ainda não existirem (eles nascem dos `.org`), gere-os antes:

```
make nb
```

Abra `01-panorama-escolas.ipynb` e, no canto superior direito, selecione o kernel **Python (censo-escolar)**. Se o notebook abrir já com outro kernel, use `Kernel > Change Kernel...`

Passo 4 — rodar

Shift+Enter executa a célula e avança; **Ctrl+Enter** executa sem sair do lugar; **Run > Run All Cells** roda o documento inteiro. A ordem importa: a primeira célula faz os imports e define `ANO`, e todas as outras dependem dela.

A carga completa de um ano leva alguns segundos (bem menos depois do `make parquet`). Se quiser começar leve, restrinja por UF:

```
escolas = carregar_escolas(ANO, uf="MG")
```

Passo 5 — o ciclo de desenvolvimento

Aqui está a parte que muda em relação a um notebook comum. **Nada de lógica nova dentro de célula.** O ciclo é:

1. Você explora no notebook até a análise funcionar.
2. Assim que virar algo reutilizável (uma agregação, um recorte, um gráfico), move para `src/censo_escolar/` — `analysis.py` para agregações, `plots.py` para gráficos, `loading.py` para leitura.
3. Exporta o nome em `src/censo_escolar/__init__.py`.
4. Escreve o teste em `tests/`.
5. A célula do notebook fica com uma linha: a chamada.

Para não reiniciar o kernel a cada edição do pacote, ligue o autoreload na primeira célula:

```
%load_ext autoreload
%autoreload 2
```

Com isso, salvar um `.py` em `src/censo_escolar/` já reflete na próxima execução da célula. O autoreload não recupera tudo (mudança de assinatura de classe, por exemplo) — quando o comportamento ficar estranho, `Kernel > Restart Kernel and Run All Cells` resolve.

Rode os testes do lado de fora, sem fechar o Lab:

```
make test
make lint
```

Passo 6 — gráficos

No Jupyter, chame a função de plot **sem** o argumento `arquivo` e a figura aparece inline:

```
plots.aplicar_estilo() # uma vez por sessão
plots.barras(uf.head(15), x="SG_UF", y="escolas", titulo="Escolas por UF")
```

Com `arquivo`, ela grava o PNG em `reports/figures/` e devolve o caminho como string — é a forma que o org-babel precisa. Se você mantiver o `arquivo` no notebook, verá o caminho impresso em vez da imagem; isso é esperado e é o preço de a mesma célula servir aos dois ambientes.

Para exibir e gravar, capture o eixo:

```
ax = plots.barras(uf.head(15), x="SG_UF", y="escolas")
ax.figure.savefig("../reports/figures/escolas_por_uf.png")
```

Passo 7 — sincronizar e commitar

O JupyterLab roda com o diretório de trabalho em `notebooks/`, mas todo caminho do pacote é resolvido a partir da raiz do projeto (`config.encontrar_raiz()`), então `carregar_escolas` e `plots` acham os arquivos independentemente de onde o kernel foi iniciado.

Antes de commitar, propague suas mudanças do `.ipynb` para o `.org`:

```
make sync          # decide a direção pelo mtime - aqui, .ipynb -> .org
make check-sync    # confere; sai com código 1 se algo ficou pendente
git add notebooks/ src/ && git commit -m "..."
```

Salve o notebook no Lab antes de rodar o sync (Ctrl+S): o mtime do arquivo em disco é o que decide quem manda. E não edite o `.org` do mesmo par em paralelo — o sync sobrescreve o lado mais antigo, não faz merge.

Executando sem interface

Útil em CI ou para regenerar as saídas de todos os notebooks:

```
.venv/bin/jupyter nbconvert --to notebook --execute --inplace \
    --ExecutePreprocessor.kernel_name=censo-escolar \
    notebooks/01-panorama-escolas.ipynb

.venv/bin/jupyter nbconvert --to html notebooks/01-panorama-escolas.ipynb
```

Depois de um `--execute --inplace`, rode `make sync` para o `.org` acompanhar.

Quando algo dá errado

Sintoma	Causa provável e saída
<code>ModuleNotFoundError: censo_escolar</code>	Kernel errado. Troque para Python (censo-escolar) ; s
<code>FileNotFoundError</code> ao carregar o ano	Microdados não baixados: <code>make dados ANO=2023</code>
<code>ConnectionResetError</code> no download	Instabilidade do INEP; já há retentativa. Tente de novo
<code>SSL_ERROR / local issuer certificate</code>	Cadeia incompleta do INEP: <code>make certificados</code>
Kernel morre na carga	Memória. Use <code>uf="MG"</code> ou reduza <code>colunas</code> ; rode <code>make pa</code>
Notebook não aparece no Lab	Ele é gerado do <code>.org</code> ; rode <code>make nb</code>
Mudei o <code>.py</code> e o notebook não viu	Falta <code>%autoreload 2</code> , ou reinicie o kernel
<code>make check-sync</code> falha no commit	Salve o notebook (<code>Ctrl+S</code>) e rode <code>make sync</code>
Alterações no <code>.ipynb</code> sumiram	O par <code>.org</code> estava mais novo e o <code>sync</code> sobrescreveu — ve

O fluxo de sincronização

```

make sync          # converte cada par na direção do arquivo mais recente
make check-sync    # não escreve nada; sai com código 1 se houver pendências
make nb            # força .org -> .ipynb em todos os documentos
make org           # força .ipynb -> .org em todos os documentos

```

Regras de convivência

Estas três regras evitam praticamente todo conflito:

1. **Rode `make sync` antes de commitar.** Os dois arquivos do par vão para o git; o `make check-sync` é o que você chamaria numa CI ou num hook de pre-commit.
2. **Não edite os dois lados do mesmo par ao mesmo tempo.** O `sync` é decidido por `mtime` e sobrescreve o lado mais antigo — ele não faz merge.
3. **Lógica nova vai para `src/censo_escolar/`, nunca para dentro de um bloco.** Documento é narrativa; pacote é código.

O que se preserva e o que não se preserva

Elemento	org → ipynb → org	ipynb → org → ipynb
Código dos blocos	idêntico	idêntico
Argumentos de cabeçalho (:session etc.)	idêntico	idêntico
Prosa, títulos, listas	idêntico	idêntico
Saídas / <code>execution_count</code>	preservados	preservados
Ids das células	estáveis	estáveis
<code>#+RESULTS:</code> do org	descartado ¹	—
Blocos de outras linguagens (<code>sh</code> , <code>elisp</code>)	idêntico ²	idêntico ²
Finais de linha	sempre <code>==</code> ³	sempre <code>==</code> ³

¹ A saída canônica passa a viver no `.ipynb`. Reexecute o bloco no Emacs para regenerar o `#+RESULTS:` localmente.

² Viram blocos de código cercados dentro de uma célula markdown — visíveis no Jupyter, mas não executáveis lá.

³ Independente do sistema operacional, de propósito: veja No Windows.

Estrutura do projeto

```
src/censo_escolar/
  config.py      caminhos do projeto, resolvidos a partir da raiz
  download.py    download e extração dos ZIPs do INEP
  loading.py     leitura dos CSVs, dtypes, cache em Parquet
  codigos.py     dicionários de códigos (TP_DEPENDENCIA, UFs, ...)
  analysis.py    agregações reutilizáveis
  plots.py       gráficos que funcionam nos dois ambientes
  orgnb.py       conversor org <-> ipynb
  cli.py         comando 'censo'
notebooks/       documentos de análise (.org e .ipynb pareados)
docs/microdados-2025.org  mudanças de formato do pacote do INEP
emacs/censo-init.el
tests/
data/{raw,interim,processed}/  ignorados pelo git
reports/figures/               ignorados pelo git
```

Por que os caminhos são resolvidos a partir da raiz

O Jupyter roda com o diretório de trabalho em `notebooks/`; o `org-babel` roda onde o Emacs estiver. Se o código usasse caminhos relativos ao `cwd`, quebraria em um dos dois. Por isso `config.encontrar_raiz()` sobe a árvore de diretórios até achar o `pyproject.toml`, e todo caminho sai de lá.

Referência rápida da API

```
from censo_escolar import (
    carregar_escolas, carregar_anos, colunas_disponiveis, converter_para_parquet,
    panorama_por_uf, distribuicao_dependencia, taxa_infraestrutura,
    contar_escolas, somar_matriculas, serie_historica, maiores_municipios,
    rotular, adicionar_regiao, get_paths,
)

# Carga: por padrão só escolas em atividade, com um subconjunto útil de colunas
escolas = carregar_escolas(2023, uf="MG")
escolas = carregar_escolas(2023, colunas=colunas_disponiveis(2023)) # tudo

# Agregações - todas recebem e devolvem DataFrame, sem imprimir nem plotar
panorama_por_uf(escolas) # escolas, matrículas, docentes por UF
distribuicao_dependencia(escolas, por="SG_UF") # participação de cada rede
taxa_infraestrutura(escolas, por="SG_UF") # % de escolas com cada item IN_*
maiores_municipios(escolas, n=20)
serie_historica(carregar_anos([2019, 2021, 2023]), por="SG_UF")

# Códigos -> texto, sem perder a coluna original (cria DS_DEPENDENCIA)
rotular(escolas, "TP_DEPENDENCIA")
```

Colunas ausentes no ano são ignoradas em vez de causar erro: o INEP renomeia e aposenta variáveis entre edições, e travar por causa disso inviabilizaria comparações entre anos.

Tipos e os códigos alfanuméricos do INEP

Os dtypes são escolhidos pelo prefixo da coluna (IN_ e TP_ viram Int8, QT_ vira Int32, CO_ vira Int64, NO_!=SG_!=DS_ viram string). É o que segura o consumo de memória num arquivo de ~370 colunas.

Só que o prefixo mente em alguns casos: CO_ORGAO_REGIONAL, apesar do CO_, traz valores como OMI11 em parte dos estados. Quando a conversão falha, a leitura **não quebra** — o CSV é relido em modo tolerante, cada coluna é convertida individualmente e as que não forem numéricas de fato permanecem texto, com um aviso dizendo quais foram:

UserWarning: Colunas lidas como texto por conterem valores não numéricos: CO_ORGAO_REG

Repare que o fallback preserva o valor original em vez de aplicar um `errors="coerce"`, que transformaria `=0MI11` em nulo silenciosamente — e também preserva os zeros à esquerda (`00009`), que um `Int64` comeria.

Desenvolvimento

```
make test    # pytest
make lint    # ruff
```

Alternativas consideradas

Ferramenta	Situação
jupytertext	A escolha óbvia — mas não suporta org-mode (só md, MyST, Rmd, Quarto, py)
pandoc	Converte org ipynb, porém descarta os argumentos de cabeçalho
ox-ipynb	Só exporta org → ipynb; não há caminho de volta
ob-ipython	Descontinuado em favor do <code>emacs-jupyter</code>

Daí o `orgnb.py`: cerca de 500 linhas, sem dependências externas, e preservando justamente o que as outras opções perdem. Se um dia o `jupytertext` ganhar suporte a org, migrar é trocar um alvo do Makefile.

Vale notar que `jupytertext` continua útil aqui como complemento: se alguém do time preferir trabalhar em `py:percent`, dá para parear o `.ipynb` com um `.py` sem interferir no fluxo do org.

Licença

MIT.